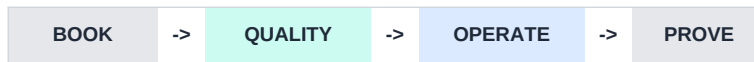


C3-08 Quality, Observability, Security & DataOps

Beginner-first canonical learner book | 15 lessons | 44.5 lesson-hours



2026 LEARNER EDITION

Option B institute family: Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6 | AMOLED-ready companion shell.

How to use this book

For every lesson: understand the picture -> define new words -> follow the worked example -> reproduce the guided version -> close the book -> do the matching practice alone -> validate -> save evidence -> only then open the protected review if needed -> close it -> solve the changed retrv.

External media is support-only. The book + starter project contain enough instruction to complete the work. Do not mark a lesson mastered from reading or watching alone.

This book is your teacher

Use this book as the primary teacher. External videos and official documentation are visual or current-behavior companions, never hidden prerequisites.

The locked beginner-first lesson contract

- | | |
|-----------------------------------|---|
| 01. Why it matters | 09. Why it worked |
| 02. What you learn | 10. Common beginner mistakes |
| 03. Prerequisite / starting state | 11. If stuck / recovery |
| 04. Picture it in your head | 12. Small independent task |
| 05. New words before use | 13. Validate |
| 06. Useful video/source | 14. Teach it back |
| 07. Follow with me once | 15. Protected review + changed retry |
| 08. Expected result | 16. Evidence/mastery + exact next route |

Rules that protect your learning

- Save a genuine attempt before opening a protected review.
- After review, close it and solve a changed retry - do not copy the reviewed answer.
- A command that runs is not mastery: validate grain, keys, counts, state, failure/recovery and evidence.
- If a new engineering concept appears, this book explains it from first principles even when Bridge skills are assumed.
- When stuck, repair only the named weak competency; do not restart the whole phase.

Contents / Index

Actual page numbers after the master-audit repair. Mentor routes use these same pages.

Front matter	Page
This book is your teacher	2
Contents / Index	3
Module map + practice/review/Gate route	4
Phase-specific study notes	5
Lessons	Start
C3-08-L01 Data-quality dimensions and enforceable contracts	6
C3-08-L02 Blocking vs warning checks and severity design	9
C3-08-L03 Source-to-target reconciliation patterns	12
C3-08-L04 Freshness, completeness and timeliness controls	15
C3-08-L05 Metadata, lineage and ownership	18
C3-08-L06 Logs, metrics and tracing concepts	21
C3-08-L07 Pipeline SLIs, SLOs and alert design	24
C3-08-L08 Incident triage and root-cause analysis	27
C3-08-L09 Runbooks and operational handoff	30
C3-08-L10 Secrets, identity and access-control engineering	33
C3-08-L11 Encryption and sensitive-data handling	36
C3-08-L12 CI/CD for data projects and automated checks	39
C3-08-L13 Environment promotion and rollback	42
C3-08-L14 Infrastructure-as-code concepts for data systems	45
C3-08-L15 Cost, performance and reliability trade-offs	48
Mastery checklist + revision quick reference	51

Module map + protected learning route

Quality, Observability, Security & DataOps

- 01 Prerequisite / prior-phase Gate
- 02 Lesson mental model + vocabulary
- 03 Follow guided example once
- 04 Matching independent practice
- 05 Save genuine attempt
- 06 Protected review unlocks
- 07 Close review + changed retry
- 08 Validate + explain back
- 09 Master lesson + save evidence
- 10 Mini-Lab / integrated drill
- 11 Protected Gate A
- 12 If needed: protected review -> targeted repair -> changed micro-proof
- 13 Explicitly assign fresh Gate B/C only after repair evidence
- 14 Phase PASS -> next phase

What counts as mastery

- You can rebuild the competency without the lesson/review open.
- Your validation catches a plausible wrong result, not only syntax failure.
- You can state one failure mode and the recovery step.
- Your evidence names the input/output or artifact and the independent check used.

Revision rule

Revise from evidence: weak validation, weak recovery or weak explanation sends you back only to the matching lesson/repair route. Completion percentage never overrides a failed Gate.

Phase-specific study notes

C3-08 treats quality, observability, security and DataOps as production correctness. A pipeline is not healthy merely because it is running.

Use the lesson contract on pages 6 onward. For every quality or operations control, record the requirement, measurable signal, severity/owner, recovery action and independent revalidation.

Protected reviews stay closed until a genuine attempt is saved. Gate B/C are fresh full retests and require the failed prior Gate, its protected review, targeted repair evidence and an explicit reassessment assignment.

External docs may change; the durable reasoning in this book remains the learning authority. When vendor behavior is current-sensitive, verify the current official source before applying it.

C3-08-L01 - Data-quality dimensions and enforceable contracts

Focused effort: 2.5 h | Practice P01 | Review R01 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Turn vague quality expectations into an enforceable contract with grain, keys, schema, allowed domains, ownership and severity.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

Think of a data contract as the label and safety rules on a medicine bottle: what the item is, which fields must exist, what values are allowed, and who acts if the rules fail.

5. New words before we use them

Grain = what one row represents. Invariant = rule that must remain true. Domain = allowed values. Contract = machine- or human-readable agreement about structure and behavior.

6. Useful current source / visual

RECOMMENDED: OpenLineage Data Quality Assertions - Current model for assertion result + severity. Segment: whole relevant page.

[Open mapped source](#)

C3-08-L01 - Follow with me once

7. Plain-English explanation

Quality is multi-dimensional: completeness, validity, uniqueness, consistency, timeliness and accuracy proxies. A contract must state one-row meaning, stable identifiers, required fields, accepted domains, known assumptions and who owns a breach. A quality check without declared grain can reject valid repeats or miss true duplicates.

8. Tiny worked example

Orders are one row per order_id. Duplicate order_id breaks the grain. A missing optional owner_name may be tolerated, but a missing order_id cannot.

9. Follow with me once

Inspect orders_contract.json. Explain why order_id is the grain key, why missing customer_id can be a warning while duplicate order_id is blocking, and why expected domain/status belongs in the contract.

10. Exact action / structure

Complete contracts/L01_orders_contract_TODO.json. Save your own artifact, command/output and independent validation.

11. Expected result

A contract that names grain, stable key, required fields, allowed status values, owners, and at least four checks with explicit consequence.

12. Why it worked

The checks are tied to the declared grain and consumer risk, so the contract can distinguish harmless missing context from corruption.

13. Break / fix / debug

Break: change the grain to order-status events. Fix: redesign the key and uniqueness rule instead of forcing one-row-per-order logic.

14. Common beginner mistakes

Writing checks before declaring grain; treating every NULL the same; using vague words like clean/good; having no owner when a rule fails.

C3-08-L01 - Do alone, validate and master

15. Recovery if stuck

If you cannot classify a rule, restate the consumer impact and table grain before choosing severity.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

Change the data to an event stream where one order legitimately has many status events; redesign the key and checks.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L01/attempt_001/ first. Only then may R01 be opened if needed. Close it and solve the changed transfer in evidence/L01/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save contract + changed-grain transfer; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L02 - Blocking vs warning checks and severity design

Focused effort: 2.5 h | Practice P02 | Review R02 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Choose block, warn or info from downstream risk and prove that critical failures stop unsafe publish.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

A quality system is a traffic signal: red stops unsafe publication, amber allows controlled continuation with ownership, green is informational context.

5. New words before we use them

Blocking = prevents publish. Warning = records a violation but permits controlled continuation. Severity = consequence of a failed check. Blast radius = who/what is affected.

6. Useful current source / visual

RECOMMENDED: OpenLineage Data Quality Assertions - Current error-vs-warn semantics. Segment: success vs severity section.

[Open mapped source](#)

C3-08-L02 - Follow with me once

7. Plain-English explanation

Severity is an operational policy, not a preference. Blocking checks protect invariants whose violation can corrupt published truth, security or recoverability. Warnings preserve visibility for lower-risk anomalies. Informational signals add context. A warning must still be observable and owned.

8. Tiny worked example

A duplicate primary business key is blocking for a finance close. A missing optional description can be warning. A row-count metric can be informational.

9. Follow with me once

Run the quality helper against the practice source. Compare a duplicate-key violation, an unknown status and a missing optional owner field. State publish behavior for each.

10. Exact action / structure

Complete quality/L02_severity_policy_TODO.md. Save your own artifact, command/output and independent validation.

11. Expected result

A severity policy that maps each check to consumer impact, publish action, owner, escalation and recovery.

12. Why it worked

Severity is derived from harm and recoverability rather than from personal preference.

13. Break / fix / debug

Break: reclassify duplicate key as warning. Fix: trace downstream double-count risk and restore it to blocking.

14. Common beginner mistakes

Making everything blocking; making everything warning; alerting without an owner; confusing test failure with pipeline failure.

C3-08-L02 - Do alone, validate and master

15. Recovery if stuck

If everything blocks, distinguish business-critical invariants from triage-worthy anomalies. If nothing blocks, identify the corruption you are allowing through.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

Reclassify one check after the consumer changes from exploratory dashboard to financial close.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L02/attempt_001/ first. Only then may R02 be opened if needed. Close it and solve the changed transfer in evidence/L02/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save severity policy + publish decision; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L03 - Source-to-target reconciliation patterns

Focused effort: 3.5 h | Practice P03 | Review R03 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Prove publication correctness with counts, keys, totals and record-level mismatch evidence instead of trusting job status.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

Reconciliation is like checking a bank transfer using more than the balance: you compare who was transferred, how many records moved, and whether important totals agree.

5. New words before we use them

Key set = identifiers present. Control total = independent aggregate used for verification. Reconciliation = comparing independent source and target evidence.

6. Useful current source / visual

RECOMMENDED: OpenLineage Object Model - Dataset/run metadata and data-quality facets. Segment: Dataset and data-quality facet portions.

[Open mapped source](#)

C3-08-L03 - Follow with me once

7. Plain-English explanation

Reconciliation compares independent source and target evidence. Count parity alone can hide one missing and one extra row. Total parity can hide offsetting errors. Strong reconciliation combines row count, key set, control totals and changed-record comparison at the declared grain.

8. Tiny worked example

Source has 8 orders; target has 7. Even if total sales happen to match, missing key O506 proves the publication is incomplete.

9. Follow with me once

Run the reference reconciliation on `source_orders.csv` and `target_orders_corrupt.csv`. It must expose one missing key and one mismatched amount, then quantify the total delta.

10. Exact action / structure

Complete `scripts/03_reconcile_TODO.py`. Save your own artifact, command/output and independent validation.

11. Expected result

A report showing source/target counts, missing/extra keys, value mismatches and control-total delta at the stated grain.

12. Why it worked

Multiple independent checks prevent offsetting errors from hiding one another.

13. Break / fix / debug

Break: add an extra target row with the same amount as a missing row. Fix: key-set reconciliation still catches the defect.

14. Common beginner mistakes

Checking only row count; checking only total; using `DISTINCT` to hide duplicate keys; comparing different grains.

C3-08-L03 - Do alone, validate and master

15. Recovery if stuck

If totals differ but you cannot explain why, compare key sets first and then row-level values for common keys.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

Add one extra target row that offsets the missing-row total; show why key-set reconciliation still catches the defect.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L03/attempt_001/ first. Only then may R03 be opened if needed. Close it and solve the changed transfer in evidence/L03/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save reconciliation report + changed offsetting-error case; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L04 - Freshness, completeness and timeliness controls

Focused effort: 3.0 h | Practice P04 | Review R04 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Convert freshness/completeness expectations into measurable SLIs with explicit observation time, expected arrivals and escalation thresholds.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

Freshness is the age of the newest trustworthy data; completeness asks whether all expected pieces arrived; timeliness compares arrival against the business deadline.

5. New words before we use them

Freshness age = observation time minus latest valid event/load time. Completeness = expected pieces received. Timeliness = met the promised deadline.

6. Useful current source / visual

RECOMMENDED: Google SRE - Service Level Objectives - SLI/SLO control-loop reasoning for timeliness/freshness. Segment: control measures + SLO design.

[Open mapped source](#)

C3-08-L04 - Follow with me once

7. Plain-English explanation

Freshness is age relative to a defined timestamp and observation point. Completeness is whether expected files/partitions/records arrived. Timeliness asks whether data arrived within the business deadline. These are not the same as pipeline runtime status. Time zones and late-arrival policies must be explicit.

8. Tiny worked example

At 08:30, newest valid partition arrived at 06:00: freshness age is 150 minutes. If the SLA is 120 minutes, the pipeline is late even if its last run is green.

9. Follow with me once

Use the fixed observation timestamp in `freshness_config.json` and calculate source freshness minutes, expected daily partition completeness and breach state.

10. Exact action / structure

Complete `slo/L04_freshness_completeness_TODO.md`. Save your own artifact, command/output and independent validation.

11. Expected result

Fixed-time freshness calculation, expected-partition completeness check, breach decision, owner and alert action.

12. Why it worked

Using a fixed observation time makes the result reproducible and separates scheduler status from consumer-facing freshness.

13. Break / fix / debug

Break: mark a late partition complete because the job succeeded. Fix: measure newest valid business timestamp and expected partition set.

14. Common beginner mistakes

Using current clock during tests; measuring job end instead of data time; ignoring expected missing partitions; paging on harmless delay.

C3-08-L04 - Do alone, validate and master

15. Recovery if stuck

If freshness changes each time you run the test, you forgot to make observation time explicit. If completeness is just row count, define expected arrival units.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

Move the SLA from 120 minutes to 180 minutes and explain the changed alert behavior without changing data.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L04/attempt_001/ first. Only then may R04 be opened if needed. Close it and solve the changed transfer in evidence/L04/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save freshness/completeness control + changed SLA; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L05 - Metadata, lineage and ownership

Focused effort: 3.0 h | Practice P05 | Review R05 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Capture dataset/job/run lineage and attach actionable technical, data and consumer ownership.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

Lineage is a metro map: datasets are stations, jobs are trains, and run metadata tells you which specific trip moved data between stations.

5. New words before we use them

Job = defined transformation. Run = one execution. Dataset = named data asset. Lineage = relationships showing how data is produced/consumed. Facet = extra metadata attached to an OpenLineage entity/event.

6. Useful current source / visual

RECOMMENDED: OpenLineage Object Model - RunEvent/JobEvent/DatasetEvent and job-run-dataset model. Segment: whole relevant page.

[Open mapped source](#)

OPTIONAL: Cross-Platform Data Lineage with OpenLineage - Visualize lineage across Airflow/Spark and a live lineage graph. Segment: Architecture + live demo; provider publishes no chapter timestamps..

[Open mapped source](#)

C3-08-L05 - Follow with me once

7. Plain-English explanation

Lineage is not a picture only. Operational lineage connects datasets, jobs and run events with producer/version/time metadata, so a change or incident can answer what ran, what changed and what is downstream. OpenLineage models run, job and dataset events; the course uses the model without requiring a lineage server.

8. Tiny worked example

orders_raw -> build_orders_curated -> orders_curated. Store the code version and run ID so an incident can identify which release created a bad dataset.

9. Follow with me once

Fill lineage/L05_lineage_event_TODO.json for source -> quality/reconcile job -> curated target and include code version, run_id and owners.

10. Exact action / structure

Complete lineage/L05_lineage_event_TODO.json. Save your own artifact, command/output and independent validation.

11. Expected result

An OpenLineage-shaped metadata event plus owner, code version, input/output dataset identifiers and downstream blast-radius note.

12. Why it worked

The event separates design metadata from one execution and creates enough context to trace ownership and impact.

13. Break / fix / debug

Break: one job reads customers+orders and independently writes two outputs; Fix: avoid implying Cartesian lineage edges.

14. Common beginner mistakes

Drawing arrows without owner/version; treating filename only as dataset identity; creating false all-to-all edges; confusing run with job.

C3-08-L05 - Do alone, validate and master

15. Recovery if stuck

If lineage has arrows but no run/version/owner, it cannot support incident diagnosis or change blast-radius decisions.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

Add a downstream daily_sales asset and identify who must be notified for a breaking schema change.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L05/attempt_001/ first. Only then may R05 be opened if needed. Close it and solve the changed transfer in evidence/L05/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save lineage event + blast-radius transfer; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L06 - Logs, metrics and tracing concepts

Focused effort: 3.0 h | Practice P06 | Review R06 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Design correlated logs, metrics and trace/span context that diagnose one run without leaking sensitive data.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

Logs are the diary, metrics are the dashboard gauges, and traces are the journey map. Correlation IDs let all three describe the same run.

5. New words before we use them

Log = discrete event record. Metric = numeric measurement over time. Trace = end-to-end path composed of spans. Resource = origin metadata. Trace/span ID = correlation context.

6. Useful current source / visual

RECOMMENDED: OpenTelemetry Overview - Logs/metrics/traces/resource/context model. Segment: signals + resource/context sections.

[Open mapped source](#)

OPTIONAL: Logs, Metrics, Traces: Correlate with OpenTelemetry - Visual debugging from logs to traces and metrics. Segment: 0:00 problem; 0:58 logs->traces; 2:39 logs->metrics; 3:47 semantic mapping..

[Open mapped source](#)

C3-08-L06 - Follow with me once

7. Plain-English explanation

OpenTelemetry provides a vendor-neutral observability model. Logs capture discrete events, metrics summarize behavior over time and traces connect work across components. Correlation fields such as `run_id`, `trace_id`, `dataset` and `code version` make separate signals usable together. High-cardinality or sensitive fields should not be dumped blindly into telemetry.

8. Tiny worked example

A failed run logs `run_id=R42` and `trace_id=T9`; metric `failed_rows=3` carries the same run context; trace T9 shows failure in `validate` step.

9. Follow with me once

Inspect `telemetry/run_events.jsonl` and `metrics.json`. Identify the `run_id/correlation` fields that let you connect quality failure, reconciliation mismatch and publish block.

10. Exact action / structure

Complete `observability/L06_signal_contract_TODO.md`. Save your own artifact, command/output and independent validation.

11. Expected result

A signal contract naming required fields for logs, metrics and trace context without raw PII or secrets.

12. Why it worked

Shared run/trace/resource context lets different telemetry signals answer complementary questions about the same execution.

13. Break / fix / debug

Break: remove `run_id` from logs. Fix: restore correlation fields and prove a responder can connect failure evidence.

14. Common beginner mistakes

Logging entire records; using metrics without labels; creating trace IDs but not logging them; putting secrets in error payloads.

C3-08-L06 - Do alone, validate and master

15. Recovery if stuck

If a log says only error=true, add operation, dataset, run_id, error class and safe context. If it contains raw PII or credentials, redesign the evidence.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

Add a second job in the same trace and state which fields should propagate across the boundary.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L06/attempt_001/ first. Only then may R06 be opened if needed. Close it and solve the changed transfer in evidence/L06/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save signal contract + correlation transfer; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L07 - Pipeline SLIs, SLOs and alert design

Focused effort: 3.0 h | Practice P07 | Review R07 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Define service indicators and objectives that measure data reliability and create actionable alerts rather than noise.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

An SLI is the speedometer; an SLO is the target range; an alert is the rule for when a human must act.

5. New words before we use them

SLI = measured reliability indicator. SLO = target for an SLI over a window. Alert = actionable notification. Error budget = tolerated unreliability implied by the target.

6. Useful current source / visual

RECOMMENDED: Google SRE - Implementing SLOs - SLO design and measurement. Segment: SLO definition/implementation sections.

[Open mapped source](#)

OPTIONAL: OpenTelemetry Explained for Beginners - Beginner visual for metrics/logs/traces before SLI design. Segment: Topic-directed; provider metadata exposes no chapters..

[Open mapped source](#)

C3-08-L07 - Follow with me once

7. Plain-English explanation

An SLI is a measured behavior such as on-time partition ratio, successful validated publish ratio, freshness minutes or reconciliation pass ratio. An SLO sets an objective over a window. Alerts should tie to user impact, burn/risk and a clear responder action. Every transient warning does not need paging.

8. Tiny worked example

Validated-publish SLI = 98 successful validated publishes / 100 expected runs = 98%. If SLO is 99%, the service is outside target.

9. Follow with me once

Use slo/sli_history.csv to calculate validated-publish success ratio and identify whether the supplied SLO is breached. Draft one page alert and one ticket-level alert.

10. Exact action / structure

Complete slo/L07_sli_slo_alerts_TODO.md. Save your own artifact, command/output and independent validation.

11. Expected result

At least two pipeline SLIs, explicit SLO windows, page/ticket thresholds, owner and recovery action.

12. Why it worked

The alert is tied to user-impacting reliability evidence rather than to every internal warning.

13. Break / fix / debug

Break: page on any single warning. Fix: tie paging to sustained user-impact/SLO risk and route lower severity to ticket/dashboard.

14. Common beginner mistakes

Choosing 100% by instinct; alerting on raw CPU only; SLO without time window; pager without action.

C3-08-L07 - Do alone, validate and master

15. Recovery if stuck

If your alert has no owner or action, it is only a notification. If your SLO is 100% by default, explain the operational/cost implication.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

Change one failed run to a warning-only anomaly and decide whether the page still fires.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L07/attempt_001/ first. Only then may R07 be opened if needed. Close it and solve the changed transfer in evidence/L07/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save SLO calculation + changed incident class; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L08 - Incident triage and root-cause analysis

Focused effort: 3.5 h | Practice P08 | Review R08 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Operate detect -> contain -> diagnose -> repair -> reconcile -> prevent without confusing correlation with root cause.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

Incident response is a fire drill: first contain harm, then diagnose, repair, restore, verify, and only afterward prevent recurrence.

5. New words before we use them

Containment = limit impact. RCA = root-cause analysis. Contributing factor = condition that enabled/worsened failure. Mitigation = reduce impact before permanent fix.

6. Useful current source / visual

RECOMMENDED: Google SRE Workbook - Incident Response - Containment, coordination and recovery. Segment: incident response chapter.

[Open mapped source](#)

OPTIONAL: Logs, Metrics, Traces: Correlate with OpenTelemetry - See correlated evidence used during diagnosis. Segment: 0:58-3:47 debugging/correlation flow..

[Open mapped source](#)

C3-08-L08 - Follow with me once

7. Plain-English explanation

Incident response separates observed symptom, impact, containment, evidence, contributing factors and root cause. A root cause statement must be supported by timeline and technical evidence. Recovery is incomplete until the repaired scope is reprocessed and reconciled. Prevention changes tests, contracts, release controls or runbooks.

8. Tiny worked example

A new transform corrupts amounts. Stop publish, preserve evidence, identify affected partition/release, repair only affected scope, reconcile, then add a regression check.

9. Follow with me once

Use incident/evidence_bundle.json to build a timeline for the corrupt target. State containment, likely cause, contributing factor, repair scope and post-repair proof.

10. Exact action / structure

Complete incident/L08_incident_timeline_TODO.md. Save your own artifact, command/output and independent validation.

11. Expected result

Timeline, impact, containment, evidence-backed cause, contributing factors, repair/reprocess scope, post-repair reconciliation and prevention.

12. Why it worked

Containment protects consumers while evidence is preserved; reconciliation proves restoration rather than assuming the repair worked.

13. Break / fix / debug

Break: rerun all history immediately. Fix: bound affected scope from lineage/version evidence, then reprocess only necessary data.

14. Common beginner mistakes

Starting with blame; deleting evidence; reprocessing everything; closing when job turns green; writing "human error" as root cause.

C3-08-L08 - Do alone, validate and master

15. Recovery if stuck

If your root cause is "bad data", ask what mechanism admitted it. If your fix is "rerun", ask why rerun is safe and how correctness is re-proved.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

A second incident has correct data but missing telemetry; classify impact and preventive control.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L08/attempt_001/ first. Only then may R08 be opened if needed. Close it and solve the changed transfer in evidence/L08/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save incident timeline + RCA + recovery proof; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L09 - Runbooks and operational handoff

Focused effort: 3.0 h | Practice P09 | Review R09 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Write an operator runbook with prerequisites, success evidence, diagnosis, safe rerun, rollback/escalation and ownership.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

A runbook is the emergency card beside a machine: trigger, owner, safe diagnosis, stop conditions, repair, validation and escalation.

5. New words before we use them

Runbook = tested operational procedure. Handoff = transfer of ownership/context. Escalation = when/how to involve another owner. Stop condition = condition requiring you to halt an action.

6. Useful current source / visual

RECOMMENDED: Google SRE Workbook - Incident Response - Operational ownership and response structure. Segment: roles/response workflow.

[Open mapped source](#)

C3-08-L09 - Follow with me once

7. Plain-English explanation

A runbook transfers operational ownership. It should tell another engineer what to check before running, exact commands/actions, expected evidence, common failure signatures, safe rerun boundaries, validation/reconciliation, escalation and limitations. A README that only says how to start is not a runbook.

8. Tiny worked example

If freshness breach is caused by expired credential: confirm error, do not paste secret, rotate through approved store, rerun affected partition, reconcile, record incident.

9. Follow with me once

Complete runbook/L09_runbook_TODO.md for the practice reliability harness and include the corrupted-target recovery path.

10. Exact action / structure

Complete runbook/L09_runbook_TODO.md. Save your own artifact, command/output and independent validation.

11. Expected result

Operator-ready runbook containing symptoms, owner, first checks, safe commands/actions, stop conditions, recovery, validation and escalation.

12. Why it worked

The runbook converts tribal memory into repeatable, privacy-safe response with explicit proof of recovery.

13. Break / fix / debug

Break: remove escalation/contact. Fix: add owner, severity trigger and the exact evidence required before escalation.

14. Common beginner mistakes

Writing vague "restart pipeline"; no rollback/stop boundary; embedding secrets; no validation after action.

C3-08-L09 - Do alone, validate and master

15. Recovery if stuck

If another engineer cannot decide whether a rerun is safe, add state/idempotence and reconciliation guidance.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

Change the failure from missing row to expired credential and update only the relevant diagnostic/recovery branch.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L09/attempt_001/ first. Only then may R09 be opened if needed. Close it and solve the changed transfer in evidence/L09/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save runbook + changed-failure branch; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L10 - Secrets, identity and access-control engineering

Focused effort: 3.0 h | Practice P10 | Review R10 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Keep secrets out of source/evidence, prefer short-lived identity where supported and apply least privilege across automation.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

Identity is the badge; authorization is the doors that badge can open; a secret is a key that should be short-lived, scoped and auditable when unavoidable.

5. New words before we use them

IAM = identity/access management. Least privilege = only required permissions. OIDC = federated identity token exchange. Rotation = replace credential. Revocation = make credential unusable.

6. Useful current source / visual

RECOMMENDED: OWASP Secrets Management Cheat Sheet - Secret lifecycle, least privilege, rotation, revocation and auditing. Segment: lifecycle/access/auditing/OIDC sections.

[Open mapped source](#)

RECOMMENDED: GitHub Actions OIDC in cloud providers - Short-lived federated authentication instead of long-lived cloud secrets. Segment: overview + trust conditions.

[Open mapped source](#)

C3-08-L10 - Follow with me once

7. Plain-English explanation

Secrets need centralized storage, controlled provisioning, rotation, audit, revocation and break-glass planning. GitHub Actions can use environment protections and OIDC to avoid long-lived cloud credentials. Workflow permissions should be minimal. Repository secrets are not a substitute for identity design.

8. Tiny worked example

A CI job needs read repo + write one deployment target. With OIDC it exchanges a short-lived identity token instead of storing a long-lived cloud key in GitHub.

9. Follow with me once

Inspect `.github/workflows/ci_TODO.yml` and `security/L10_access_matrix_TODO.md`. Identify required workflow permissions, protected environment boundary and where OIDC would replace a stored cloud key.

10. Exact action / structure

Complete `security/L10_access_matrix_TODO.md`. Save your own artifact, command/output and independent validation.

11. Expected result

Least-privilege matrix, minimal workflow permissions, protected production environment, secret lifecycle and OIDC trust boundary.

12. Why it worked

Short-lived federated credentials reduce stored-secret exposure, while environment protections constrain who/what can reach production.

13. Break / fix / debug

Break: allow any fork/branch to request production token. Fix: constrain trust claims and protected environment/branch rules.

14. Common beginner mistakes

Giving admin to make pipeline work; repo-wide long-lived secret; logging tokens; OIDC trust policy without branch/repo conditions.

C3-08-L10 - Do alone, validate and master

15. Recovery if stuck

If a workflow needs broad write-all permissions, enumerate the exact API/action that needs each permission. If a secret appears in logs, treat it as compromised and rotate/revoke.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

Move deployment from test to production and add required reviewer/environment restrictions.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L10/attempt_001/ first. Only then may R10 be opened if needed. Close it and solve the changed transfer in evidence/L10/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save access matrix + OIDC/protected-environment design; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L11 - Encryption and sensitive-data handling

Focused effort: 2.5 h | Practice P11 | Review R11 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Classify sensitive data, minimize exposure and produce privacy-safe evidence while distinguishing encryption, masking and access control.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

Sensitive data handling is a chain of custody: classify first, then minimize, mask/encrypt, restrict, retain only as long as needed, and sanitize evidence.

5. New words before we use them

Encryption at rest = stored-data protection. Encryption in transit = protected network transport. Masking = hide part/all of a value. Classification = sensitivity category. Minimization = collect/store only what is needed.

6. Useful current source / visual

RECOMMENDED: OWASP Secrets Management Cheat Sheet - Sensitive secret handling, TLS, auditing and incident response. Segment: TLS/logging/incident response sections.

[Open mapped source](#)

RECOMMENDED: OneLake Security Overview - Current data-plane access context for Fabric evidence. Segment: whole relevant page.

[Open mapped source](#)

C3-08-L11 - Follow with me once

7. Plain-English explanation

Encryption protects data in transit/at rest; access control decides who may use it; masking/redaction reduces what is exposed in outputs/evidence. None replaces the others. Operational screenshots/logs should use synthetic or sanitized values and avoid secret, token, personal email or tenant-private identifiers unless strictly required and protected.

8. Tiny worked example

Incident evidence needs order_id and error code, not full customer email, token, or payment detail. Replace email with a synthetic/sanitized identifier.

9. Follow with me once

Run the redaction helper on evidence/pii_sample.json. Confirm email/token-like fields are masked before evidence is saved.

10. Exact action / structure

Complete security/L11_sensitive_evidence_policy_TODO.md. Save your own artifact, command/output and independent validation.

11. Expected result

Classification, encryption, access, masking, evidence-sanitization and retention rules plus one negative exposure test.

12. Why it worked

Controls are selected from data sensitivity and purpose, reducing both breach impact and accidental leakage in operational evidence.

13. Break / fix / debug

Break: include raw email/token in incident bundle. Fix: remove/mask values, rotate exposed secret if applicable, preserve only safe evidence.

14. Common beginner mistakes

Assuming encryption grants authorization; copying raw PII into tickets; keeping secrets in logs; indefinite retention.

C3-08-L11 - Do alone, validate and master

15. Recovery if stuck

If you call masking encryption, restate the threat model. If raw sensitive values are required for diagnosis, define secure access and retention rather than copying them to a report.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

A regulator now requires 30-day incident evidence retention; update storage/access/expiry rules.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L11/attempt_001/ first. Only then may R11 be opened if needed. Close it and solve the changed transfer in evidence/L11/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save sanitized evidence + retention transfer; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L12 - CI/CD for data projects and automated checks

Focused effort: 3.5 h | Practice P12 | Review R12 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Build a pull-request CI gate and protected deployment path that fails unsafe data/code changes before production.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

CI is the factory inspection line; CD/promotion is the loading dock. Unsafe artifacts must fail inspection before they can reach a protected environment.

5. New words before we use them

CI = automated validation of changes. CD = controlled delivery/deployment. Workflow = automation definition. Environment = named deployment target with protections/secrets. Gate = required check before promotion.

6. Useful current source / visual

RECOMMENDED: GitHub Deployments and Environments - Deployment protection rules, environment secrets and branches. Segment: protection rules + environment secrets.

[Open mapped source](#)

OPTIONAL: Continuous delivery with GitHub Actions - Official GitHub visual for protected environments and scoped secrets. Segment: 3:55 protect environments; 5:36 scope secrets; 7:02 deployment tracking; 8:44 demo..

[Open mapped source](#)

C3-08-L12 - Follow with me once

7. Plain-English explanation

CI should run syntax/unit/data-contract/reconciliation smoke checks on reviewed code. Deployment jobs should reference protected environments, use minimal permissions, serialize production deployment if needed and perform post-deploy validation. OIDC is preferable to long-lived provider secrets where supported.

8. Tiny worked example

Pull request runs syntax, unit and data-contract tests without production secrets. Main branch can request production deployment only after environment protection rules pass.

9. Follow with me once

Complete `.github/workflows/ci_TODO.yml` so tests/static checks run on pull requests and a production deploy job is environment-protected and depends on CI success.

10. Exact action / structure

Complete `cicd/L12_ci_release_checklist_TODO.md`. Save your own artifact, command/output and independent validation.

11. Expected result

A CI workflow/checklist where failing tests block promotion and PR jobs cannot access production credentials.

12. Why it worked

Validation and deployment permissions are separated, reducing both accidental bad releases and credential exposure.

13. Break / fix / debug

Break: remove a required test from ``needs`/gate`. Fix: restore dependency so deploy job cannot run after failed validation.

14. Common beginner mistakes

Deploying on every PR; storing prod secret at repo scope; ignoring failed tests; letting deployment start before approval.

C3-08-L12 - Do alone, validate and master

15. Recovery if stuck

If deployment can run after failed tests, fix dependencies. If PR workflows can access production secrets, redesign the environment/trigger boundary.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

Add a schema-contract change and define the required migration/reconciliation gate before deployment.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L12/attempt_001/ first. Only then may R12 be opened if needed. Close it and solve the changed transfer in evidence/L12/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save workflow + protected release checklist; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L13 - Environment promotion and rollback

Focused effort: 3.0 h | Practice P13 | Review R13 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Promote the same reviewed artifact through environments and choose rollback vs roll-forward based on data/schema/state compatibility.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

Promotion is moving the same sealed package through rooms with different configuration. Rollback swaps to a compatible prior package; roll-forward repairs when state/schema has moved on.

5. New words before we use them

Promotion = move reviewed artifact across environments. Rollback = return to previous version. Roll-forward = deploy corrective version. Smoke test = quick post-deploy health check.

6. Useful current source / visual

RECOMMENDED: GitHub Deployment Environments - Protected dev/staging/production targets. Segment: whole relevant page.

[Open mapped source](#)

OPTIONAL: GitHub Changelog: Manage and deploy environments - Short official visual for environments/protections. Segment: Whole 2021 demo; current behavior controlled by docs..

[Open mapped source](#)

C3-08-L13 - Follow with me once

7. Plain-English explanation

Dev/Test/Prod should differ through configuration and identity, not unreviewed code forks. Rollback is safe only if the previous version understands current schema/state and undoing side effects is bounded. Otherwise a targeted roll-forward can be safer. Every promotion needs post-deploy smoke and reconciliation evidence.

8. Tiny worked example

Release changed rounding and already wrote rows. Rolling back code may not undo written data; a bounded corrective roll-forward plus reprocessing can be safer.

9. Follow with me once

Use release/L13_promotion_rollback_TODO.md for a transform that changed numeric rounding and already wrote production rows. Decide rollback vs roll-forward.

10. Exact action / structure

Complete release/L13_promotion_rollback_TODO.md. Save your own artifact, command/output and independent validation.

11. Expected result

Dev/Test/Prod promotion plan, configuration/identity boundaries, rollback-vs-roll-forward decision, smoke test and reconciliation.

12. Why it worked

The decision considers persistent side effects and schema/state compatibility rather than treating rollback as universally safe.

13. Break / fix / debug

Break: rollback incompatible schema. Fix: choose roll-forward or compatible restore plan and validate affected data.

14. Common beginner mistakes

Different code forks per environment; no post-deploy check; rollback after irreversible data change; manual prod edits not recorded.

C3-08-L13 - Do alone, validate and master

15. Recovery if stuck

If rollback is your automatic answer, check schema/state compatibility and whether data already changed. If roll-forward, bound affected scope and prove repair.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

The release changed only stateless validation thresholds; re-evaluate the decision.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L13/attempt_001/ first. Only then may R13 be opened if needed. Close it and solve the changed transfer in evidence/L13/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save promotion plan + rollback/roll-forward decision; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L14 - Infrastructure-as-code concepts for data systems

Focused effort: 2.5 h | Practice P14 | Review R14 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Explain desired configuration, plan review, apply, state and drift without treating IaC state as ordinary source code.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

Infrastructure as code is a blueprint plus inventory: configuration says desired state, the plan shows proposed differences, and state tracks what Terraform manages.

5. New words before we use them

Plan = preview of proposed actions. State = Terraform mapping of managed objects. Backend = state storage/locking mechanism. Drift = real infrastructure differs from configuration/state. Lock = prevents concurrent state writers.

6. Useful current source / visual

RECOMMENDED: Terraform plan - Execution plan is a preview, not an apply. Segment: intro + plan behavior.

[Open mapped source](#)

RECOMMENDED: Terraform state locking - Automatic locking where backend supports it; force-unlock caution. Segment: whole relevant page.

[Open mapped source](#)

C3-08-L14 - Follow with me once

7. Plain-English explanation

Terraform plan previews proposed actions by comparing configuration and state/current objects. State can contain sensitive metadata and write operations may require locking to prevent concurrent corruption. A clean plan is evidence of intended convergence, not proof that runtime data is correct. This course keeps IaC local/conceptual and does not provision paid infrastructure.

8. Tiny worked example

A manual setting changes in production. A later plan should show drift. Review the proposed repair before apply; do not disable locking just to bypass contention.

9. Follow with me once

Inspect `iac/main.tf` and `iac/plan_review_TODO.md`. State what a speculative plan would prove, what state must be protected and how manual drift should surface.

10. Exact action / structure

Complete `iac/plan_review_TODO.md`. Save your own artifact, command/output and independent validation.

11. Expected result

Learning-only plan review explaining desired config, proposed actions, state sensitivity, locking, drift and safe review boundary.

12. Why it worked

Declarative config + state comparison exposes unmanaged differences, while locking protects state from concurrent writers where supported.

13. Break / fix / debug

Break: use `-lock=false` during shared write. Fix: resolve lock ownership safely and keep locking enabled.

14. Common beginner mistakes

Treating plan as applied result; committing sensitive state; force-unlock without verifying owner; manual fix without updating code.

C3-08-L14 - Do alone, validate and master

15. Recovery if stuck

If your plan file/state is committed casually, treat that as a security/operations defect. If drift is fixed manually without updating code, you have not restored declarative control.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

A manual production change creates one drifted setting; describe plan/review/repair without applying real cloud resources.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L14/attempt_001/ first. Only then may R14 be opened if needed. Close it and solve the changed transfer in evidence/L14/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save plan review + drift transfer; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

C3-08-L15 - Cost, performance and reliability trade-offs

Focused effort: 3.0 h | Practice P15 | Review R15 stays locked until a genuine attempt is saved.

1. Why this matters

Production data engineering includes data integrity, diagnosis, safe change, security and recovery as correctness - not as optional operations work.

2. What you learn

Make explicit architecture trade-offs across correctness, latency, reliability, operator effort and cost using evidence.

3. Prerequisite

C3-07 Gate PASS. Prior platform/data-pipeline skills are assumed, but this new production-operations concept is taught from first principles.

4. Picture it in your head

Reliability architecture is a triangle of outcome, cost and complexity. You choose a point deliberately and record what evidence would make you revisit it.

5. New words before we use them

Trade-off = gain in one dimension with cost in another. Headroom = spare capacity. ADR = architecture decision record. Review trigger = measurable condition requiring reconsideration.

6. Useful current source / visual

RECOMMENDED: Google SRE - Monitoring Distributed Systems - Monitoring simplicity, user impact and operational trade-offs. Segment: why monitor + simplicity/golden signals.

[Open mapped source](#)

C3-08-L15 - Follow with me once

7. Plain-English explanation

Optimization is multi-objective. More frequent checks can reduce detection time but cost compute; tighter SLOs can improve user experience but require redundancy/headroom; aggressive caching can reduce latency but complicate freshness. Never trade away correctness or security silently. Record the chosen option, evidence, rejected alternatives and review trigger.

8. Tiny worked example

Full reconciliation every run catches all mismatches but costs more. Sampling costs less but can miss corruption. Critical financial data may justify full reconciliation.

9. Follow with me once

Complete `adr/L15_tradeoff_ADR_TODO.md` for reconciliation every run vs sampled reconciliation, including cost, detection coverage, incident risk and review trigger.

10. Exact action / structure

Complete `adr/L15_tradeoff_ADR_TODO.md`. Save your own artifact, command/output and independent validation.

11. Expected result

ADR comparing options across correctness, latency, reliability, operator effort and cost, with non-negotiable security/correctness floors and review trigger.

12. Why it worked

The decision makes assumptions and rejected alternatives visible, so future workload changes can trigger rational review.

13. Break / fix / debug

Break: traffic doubles and sampled checks miss a defect. Fix: revisit ADR using changed risk/cost evidence.

14. Common beginner mistakes

Optimizing only cloud bill; assuming maximum redundancy is always best; hiding correctness trade-offs; no review trigger.

C3-08-L15 - Do alone, validate and master

15. Recovery if stuck

If the cheapest option wins automatically, quantify failure cost and SLO impact. If the most reliable option wins automatically, quantify marginal benefit and ongoing operational cost.

16. Guided rebuild

Rebuild from an explicit contract: requirement -> measurable control -> evidence -> severity/owner -> recovery -> independent revalidation -> preventive control. Keep credentials and raw sensitive values out of course evidence.

17. Small independent version

Traffic doubles while business freshness requirement stays unchanged; revisit the ADR.

18. Validate independently

Use an independent control appropriate to the lesson: key/totals reconciliation, fixed-time freshness, SLI calculation, correlated run evidence, negative access test, CI dependency, state/plan review or post-repair reconciliation.

19. Production / architecture reasoning

Connect the control to five questions: what consumer harm is prevented, what evidence detects failure, who owns the response, what recovery is safe, and what preventive change reduces recurrence.

20. Teach it back

Explain in 60-90 seconds what can fail, how the failure is detected, who owns it, what recovery is safe, and how you prove the system is trustworthy again.

21. Protected review + changed retry

Save evidence/L15/attempt_001/ first. Only then may R15 be opened if needed. Close it and solve the changed transfer in evidence/L15/retry_001/. Never overwrite the first attempt.

22. Evidence / mastery

Save ADR + changed-load transfer; include confidence 1-5, main uncertainty, independent validation and one sanitized error/recovery note. Mastery requires the artifact + observed result/runtime status + independent validation + one break/fix note + your own-words explanation.

Mastery checklist + revision quick reference

- ☐ Every lesson: guided rebuild completed, genuine independent attempt saved, validation recorded.
- ☐ Protected review opened only after the attempt; changed retry solved after closing the review.
- ☐ Explain-back: business/engineering purpose, grain or state, one failure mode, one recovery step, one validation check.
- ☐ Mini-Lab / integrated drill completed with reproducible evidence.
- ☐ Gate A attempted independently. If NEEDS_REPAIR: review viewed, targeted repair completed, changed micro-proof recorded, then a fresh B/C assigned explicitly.
- ☐ No unresolved critical failure, secret/private data leakage, fabricated live-run evidence or copied Gate solution.
- ☐ Evidence is saved with enough context for another person to reproduce or inspect it.

Revision quick reference

Wrong result / wrong grain	Return to validation + data contract / grain lesson; reproduce smallest failing case.
Works once, rerun breaks	Return to idempotence/state/checkpoint/recovery lesson; prove clean replay.
Cannot explain failure	Return to mental model + break/fix section; write the failure boundary in your own words.
Gate NEEDS_REPAIR	Open only its protected review, repair named weak competency, solve changed micro-proof, then assign fresh retest.
Tool/vendor changed	Use the current official source for syntax/UI, but keep the book's durable engineering contract and validation.